

CSE 4334 Assignment 1 Report

Joseph Fitchet
Rodrigo Pleitez

Part 1 Critical Thinking

Challenge A:

Challenge A: The Imputation Bias Dilemma (MCAR vs. MNAR Mechanisms)

Scenario: In the UCI Online Retail dataset, over 25% of customer identifiers (CustomerID) are missing. Suppose your team discovers that missing identifiers are not random omissions, but are systematically tied to unregistered guest checkout transactions.

Critical Prompt: If a data scientist blindly applies mean/mode imputation or replaces these missing identifiers with a single global constant like 'Guest_999', what specific mathematical and behavioral biases will be introduced into a downstream machine learning model predicting Customer Lifetime Value (CLV)? Formally contrast MCAR, MAR, and MNAR in this transactional context.

Under MCAR, where $P(R | Y_{\text{obs}}, Y_{\text{mis}}) = P(R)$, missingness is unrelated to the data, so dropping these rows would be unbiased, though it would still discard 25% of real sales. However, the scenario states the missing IDs are systematically tied to guest checkout, so the data is not MCAR. We then have to decide between MAR, where $P(R | Y_{\text{obs}}, Y_{\text{mis}}) = P(R | Y_{\text{obs}})$, and MNAR, where missingness still depends on unobserved information. Observed data cannot separate the two. We classify it as MNAR because the cause, guest checkout, is not recorded anywhere in the dataset; if there were a guest column, it would be MAR. The scenario tells us these are guests, but we do not know how many distinct guests there are or whether any are repeat buyers. So instead of imputing, we derive a HasCustomerID flag that shows when we have a registered customer and when we don't, keeping every transaction without inventing a customer.

Challenge B:

Challenge B: Scaling Disparity & Geometric Distortion in Distance Space

Scenario: A data mining team constructs a 3-Nearest Neighbor (3-NN) classification model to categorize buyer behavior. Feature 1 is UnitPrice (ranging from £0.00 to £38,000.00), while Feature 2 is Quantity (ranging from -80,000 to +80,000).

Critical Prompt: If the team computes Euclidean distance $d(x,y) = \sqrt{\sum (x_k - y_k)^2}$ on raw, unscaled features, prove geometrically why one feature completely dominates the neighbor selection process. Explain why StandardScaler resolves this magnitude distortion and analyze how extreme outliers can compress the variance of normal data points.

One of the features would dominate the neighbor selection process since the 2 terms do not have similar ranges in values. Lets say we have a bin that has as mean for UnitPrice as 8000 pounds and a quantity 1000 and a second bin for UnitPrice mean is 8000 and quantity mean at 50000.

If we have an item to place in the bins with a UnitPrice of 30000 and quantity of 5000, we would have to choose a bin by distance using Euclidean distance.

Distance from item to bin 1 is 22360.7

Distance from item to bin 2 is 50089.9

Yet the mean price of items in bin 1 is 8,000,000 pounds and the mean price of bin 2 is 400,000,000 and the current transaction is 150,000,000 yet it's distance is closer to the cheaper bin

Standard Scaler would try to make a normal distribution out of the data meaning that both means are close to 0 and both std are 1 which means that neither of the attributes would dominate as their range has been normalized to similar distribution spaces.

Extreme outliers compress the variance of datapoints since as shown in Task 2 they can inflate the variance of the data making many of the values fall to similar values really close to 0.

Challenge C:

Challenge C: Binary Proximity in Sparse Transaction Spaces (SMC vs. Jaccard)

Scenario: You convert 50,000 distinct product items into an asymmetric binary transaction matrix where '1' represents an item purchase and '0' represents absence.

Critical Prompt: If you bump into a classmate at the supermarket, would you ever say: *'Our shopping baskets are 99.9% identical because we both didn't buy the same 49,950 items (such as motor oil, caviar, and power drills)'*? Mathematically demonstrate why the Simple Matching Coefficient (SMC) is fatally flawed for sparse market-basket transactions due to mutual absence inflation (f_{00}), and prove why the **Jaccard Coefficient** is the mathematically sound metric for capturing true consumer affinity.

The primary issue with the Simple Matching Coefficient is that mutual absence (f_{00} , items neither customer bought) is counted as agreement: $SMC = (f_{11} + f_{00})/n$. In sparse baskets f_{00} dominates, so SMC severely overestimates similarity. In the supermarket example, $n = 50,000$ and $f_{00} = 49,950$. If the two shoppers share none of the remaining 50 items, $SMC = 49,950/50,000 = 0.999$, so the baskets look "99.9% identical" despite having nothing in common. This can be avoided by using the Jaccard Coefficient, $J = f_{11}/(f_{11} + f_{10} + f_{01})$, which leaves mutual absence out entirely. Similarity is then based on the items both customers bought relative to all the items either one bought, which gives $J = 0/50 = 0$ for the same two shoppers. More generally, with basket size m , $SMC \geq (n - 2m)/n$, which approaches 1 as n grows, while Jaccard does not depend on n at all.

Part II: AI integration Protocol & Prompt Engineering

Rodrigo Hernandez-Pleitez

I used ChatGPT Version: GPT-5.6 Luna

Prompt 1

How to properly use sklearn standardImputer to transform specific columns in a pandas dataframe?

The recommended way is to use `sklearn.impute.SimpleImputer` (there is no `StandardImputer` in scikit-learn) and assign the transformed values back to only the columns you want.

Example: Impute only specific columns

```
Python

import pandas as pd
from sklearn.impute import SimpleImputer

df = pd.DataFrame({
    'Age': [25, None, 30, None],
    'Salary': [50000, 60000, None, 80000],
    'City': ['NY', 'LA', None, 'Chicago']
})

# Columns to impute
num_cols = ['Age', 'Salary']

# Mean imputation
imputer = SimpleImputer(strategy='mean')

# Transform only those columns
df[num_cols] = imputer.fit_transform(df[num_cols])

print(df)
```

```
# transform only those columns
df[num_cols] = imputer.fit_transform(df[num_cols])

print(df)
```

Output:

Age	Salary	City
25.0	50000.0	NY
27.5	60000.0	LA
30.0	63333.3	None
27.5	80000.0	Chicago

Different strategies for different column types

For numerical and categorical columns, use separate imputers.

```
Python

from sklearn.impute import SimpleImputer

num_cols = ['Age', 'Salary']
cat_cols = ['City']

num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='most_frequent')

df[num_cols] = num_imputer.fit_transform(df[num_cols])
df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])
```

Best practice: Using ColumnTransformer

If you're building a machine learning pipeline, [ColumnTransformer](#) keeps preprocessing organized.

Best practice: Using ColumnTransformer

If you're building a machine learning pipeline, `ColumnTransformer` keeps preprocessing organized.

```
Python

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

numeric_features = ['Age', 'Salary']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), numeric_features)
    ],
    remainder='passthrough'
)

X_processed = preprocessor.fit_transform(df)
```

This approach is preferred because it prevents data leakage by fitting the imputer only on the training data and applying the learned statistics to validation/test sets.

📄 🔍 ↗️ ↻️ ⋮

Modified in code to

```
#making list of the columns we wish to transform
cols_to_transform = ["Quantity", "UnitPrice"]

#create imputer
imputer = SimpleImputer(strategy="median")

#apply transformation to the correnct columns
df[cols_to_transform] = imputer.fit_transform(df[cols_to_transform])

print(df[cols_to_transform].isnull().sum())
```

Prompt 2:

How to properly use sklearn StandardScaler and show its distribution data

`StandardScaler` standardizes numerical features so they have approximately **mean = 0** and **standard deviation = 1**. After scaling, you can visualize the distribution using `matplotlib` (or `pandas` plotting).

Step 1: Scale specific columns

```
Python Run

import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.DataFrame({
    'Age': [25, 30, 22, 45, 35, 28],
    'Salary': [50000, 60000, 45000, 90000, 70000, 55000]
})

# Columns to scale
cols = ['Age', 'Salary']

scaler = StandardScaler()

# Fit and transform
df_scaled = df.copy()
df_scaled[cols] = scaler.fit_transform(df[cols])

print(df_scaled)
```

Example output:

Age	Salary
-0.76	-0.76

Example output:

Age	Salary
-0.76	-0.83
-0.15	-0.17
-1.07	-1.16
1.68	1.82
0.46	0.49
-0.15	-0.17

Step 2: Check the statistics

`StandardScaler` stores the learned parameters.

`</>` Python



Run

```
print("Means:", scaler.mean_)  
print("Standard deviations:", scaler.scale_)
```

Verify the transformed data:

`</>` Python



Run

```
print(df_scaled[cols].mean())  
print(df_scaled[cols].std(ddof=0)) # Population standard deviation
```

Expected output:

Age	0.0
Salary	0.0



Age	1.0
Salary	1.0



Step 3: Visualize the distributions

Histogram before and after scaling

```
</> Python

import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(10, 6))

# Before scaling
df['Age'].hist(ax=axes[0,0], bins=8)
axes[0,0].set_title("Age (Original)")

df['Salary'].hist(ax=axes[0,1], bins=8)
axes[0,1].set_title("Salary (Original)")

# After scaling
df_scaled['Age'].hist(ax=axes[1,0], bins=8)
axes[1,0].set_title("Age (Standardized)")

df_scaled['Salary'].hist(ax=axes[1,1], bins=8)
axes[1,1].set_title("Salary (Standardized)")

plt.tight_layout()
plt.show()
```

This produces four histograms comparing the original and standardized distributions.

Step 4: Compare with a density plot (optional)

You can also overlay the distributions:

```
</> Python

fig, axes = plt.subplots(1, 2, figsize=(10,4))

df['Age'].plot(kind='density', ax=axes[0], label='Original')
df_scaled['Age'].plot(kind='density', ax=axes[0], label='Scaled')
axes[0].set_title("Age Distribution")
```

Step 4: Compare with a density plot (optional)

You can also overlay the distributions:

```
Python

fig, axes = plt.subplots(1, 2, figsize=(10,4))

df['Age'].plot(kind='density', ax=axes[0], label='Original')
df_scaled['Age'].plot(kind='density', ax=axes[0], label='Scaled')
axes[0].set_title("Age Distribution")
axes[0].legend()

df['Salary'].plot(kind='density', ax=axes[1], label='Original')
df_scaled['Salary'].plot(kind='density', ax=axes[1], label='Scaled')
axes[1].set_title("Salary Distribution")
axes[1].legend()

plt.tight_layout()
plt.show()
```

What changes after StandardScaler ?

- **Mean** becomes approximately 0.
- **Standard deviation** becomes approximately 1.
- The **shape** of the distribution does **not** change—if the original data is skewed, it remains skewed; it is simply shifted and rescaled.

📄 📌 ⬆️ ↺️ ⋮ 📖 Sources

Code implemented

```
#first we import the normalization functions for the sake of not cluttering my code
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

✓ 0.0s MagicPython

#we now have to transform the data using normalization formulas
# Making copies for later comparison
df_standard = df.copy()
df_minmax = df.copy()
df_robust = df.copy()

# we then transform them using the normalization preprocessing in the order mentioned in the assignment
df_standard[cols_to_transform] = StandardScaler().fit_transform(df[cols_to_transform])
df_minmax[cols_to_transform] = MinMaxScaler().fit_transform(df[cols_to_transform])
df_robust[cols_to_transform] = RobustScaler().fit_transform(df[cols_to_transform])

✓ 0.2s MagicPython

Once the transformations are done we have to contrast the performance of the sensitivity to extreme anomalies

print("Untransformed Dataframe")
print(df[cols_to_transform].describe())

✓ 0.0s MagicPython

Untransformed Dataframe
      Quantity  UnitPrice
count  541909.000000  541909.000000
mean      9.552250      4.611114
std     218.081158     96.759853
min    -80995.000000  -11062.000000
25%         1.000000      1.250000
50%         3.000000      2.000000
75%        10.000000      4.130000
max     80995.000000  38970.000000
```

Prompt 3

How to properly use sklearn KBinDiscretizer to discretize continues data in a panda dataframe?

`KBinsDiscretizer` converts continuous numerical columns into discrete bins. It supports three strategies:

- `uniform` – equal-width bins.
- `quantile` – equal number of samples per bin.
- `kmeans` – bins based on k-means clustering of the values.

The safest way is to apply it only to the continuous columns and then assign the transformed values back to a copy of the DataFrame.

Basic example

```
</> Python
import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer

df = pd.DataFrame({
    'Price': [5, 12, 18, 25, 40, 60, 90, 120],
    'Quantity': [1, 3, 5, 8, 12, 15, 20, 30],
    'Category': ['A', 'A', 'B', 'B', 'A', 'C', 'C', 'B']
})

# Continuous columns to discretize
continuous_cols = ['Price', 'Quantity']

# Create discretizer
discretizer = KBinsDiscretizer(
    n_bins=4,
    encode='ordinal',      # Returns bin number (0,1,2,3)
    strategy='quantile'
```

+ Ask anything

Think



```

continuous_cols = ['Price', 'Quantity']

# Create discretizer
discretizer = KBinsDiscretizer(
    n_bins=4,
    encode='ordinal',      # Returns bin numbers (0,1,2,3)
    strategy='quantile'
)

# Apply only to selected columns
df_binned = df.copy()
df_binned[continuous_cols] = discretizer.fit_transform(df[continuous_cols])

print(df_binned)

```

Output:

Price	Quantity	Category
0	0	A
0	0	A
1	1	B
1	1	B
2	2	A
2	2	C
3	3	C
3	3	B



Compare all three strategies

You can easily compare the results.

```
</> Python   Run

from sklearn.preprocessing import KBinsDiscretizer

for strategy in ['uniform', 'quantile', 'kmeans']:
    disc = KBinsDiscretizer(
        n_bins=4,
        encode='ordinal',
        strategy=strategy
    )

    result = df.copy()
    result[continuous_cols] = disc.fit_transform(df[continuous_cols])

    print(f"\n{strategy.upper()}")
    print(result[['Price', 'Quantity']])
```

How each strategy behaves

Strategy	Behavior
uniform	Splits the value range into equal-width intervals.
quantile	Each bin contains roughly the same number of observations.
kmeans	Bin boundaries are determined by k-means cluster centers.

See the actual bin boundaries

After fitting, `bin_edges_` stores the cut points.

```
</> Python   Run

disc = KBinsDiscretizer(
    n_bins=4,
    encode='ordinal',
    strategy='uniform',
    <img alt="dropdown arrow" data-bbox="461 601 488 628"/>
```

 Python



 Run

```
disc = KBinsDiscretizer(  
    n_bins=4,  
    encode='ordinal',  
    strategy='quantile'  
)  
  
disc.fit(df[continuous_cols])  
  
for col, edges in zip(continuous_cols, disc.bin_edges_):  
    print(f"{col}: {edges}")
```

Example output:

```
Price: [ 5.  16.5 32.5 67.5 120. ]  
Quantity: [ 1.  4.5 10. 16.25 30. ]
```



These edges define the intervals used to assign each observation to bins 0–3.

Visualize the discretization

A simple way to inspect the result is with a histogram of the bin assignments.

 Python



 Run

```
import matplotlib.pyplot as plt  
  
df_binned['Price'].hist(bins=4)  
plt.xlabel("Price Bin")  
plt.ylabel("Count")  
plt.title("Discretized Price (Quantile)")  
plt.show()
```



Best practice in a machine learning workflow

Best practice in a machine learning workflow

When preparing training and test data, fit the discretizer **only on the training set** and use `.transform()` on the test set.

 Python



 Run

```
disc = KBinsDiscretizer(  
    n_bins=4,  
    encode='ordinal',  
    strategy='kmeans'  
)  
  
X_train[continuous_cols] = disc.fit_transform(X_train[continuous_cols])  
X_test[continuous_cols] = disc.transform(X_test[continuous_cols])
```

This prevents data leakage by ensuring the bin boundaries are learned only from the training data.



Sources

Code Implementation

```
[13] ✓ 0.0s MagicPython  
  
#so for this part we are using only the continuous feature which is unit price so we make a copy of that  
  
df_copy = df.copy()  
prices = df_copy[["UnitPrice"]]  
  
[16] ✓ 0.0s MagicPython  
  
#reading kbinDiscretizer We have to encode with ordinal to keep it as integer  
  
uniform = KBinsDiscretizer(n_bins=12, encode="ordinal", strategy="uniform")  
quantile = KBinsDiscretizer(n_bins=12, encode="ordinal", strategy="quantile")  
kmeans = KBinsDiscretizer(n_bins=12, encode="ordinal", strategy="kmeans")  
  
[26] ✓ 0.0s MagicPython  
  
df_uniform = df.copy()  
df_quantile = df.copy()  
df_kmeans = df.copy()  
  
# we then transform them using the normalization preprocessing in the order mentioned in the assignment  
df_uniform[["UnitPrice"]] = uniform.fit_transform(df[["UnitPrice"]])  
df_quantile[["UnitPrice"]] = quantile.fit_transform(df[["UnitPrice"]])  
df_kmeans[["UnitPrice"]] = kmeans.fit_transform(df[["UnitPrice"]])  
  
[27] ✓ 0.4s MagicPython
```

Prompt 4

in KBinsDiscretizer how to show the bins and the amount of items in them?

After fitting a `KBinsDiscretizer`, you can use two attributes to inspect the bins:

- `bin_edges_` – the numerical boundaries of each bin.
- The transformed data – to count how many samples fell into each bin.

Example

```
Python

import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer

df = pd.DataFrame({
    "Price": [5, 12, 18, 25, 40, 60, 90, 120]
})

disc = KBinsDiscretizer(
    n_bins=4,
    encode="ordinal",
    strategy="quantile"
)

# Fit and transform
df["Price_bin"] = disc.fit_transform(df[["Price"]]).astype(int)

print(df)
```

Show each bin's range and count

```
Python

edges = disc.bin_edges_[0]          # Bin boundaries for the Price column
counts = df["Price_bin"].value_counts().sort_index()
```

+ Ask anything

Think



Show each bin's range and count

 Python



 Run

```
edges = disc.bin_edges_[0]          # Bin boundaries for the Price column
counts = df["Price_bin"].value_counts().sort_index()

for i in range(len(edges) - 1):
    print(f"Bin {i}: [{edges[i]:.2f}, {edges[i+1]:.2f}) -> {counts.get(i, 0)} items")
```

Example output:

```
Bin 0: [5.00, 16.50) -> 2 items
Bin 1: [16.50, 32.50) -> 2 items
Bin 2: [32.50, 67.50) -> 2 items
Bin 3: [67.50, 120.00) -> 2 items
```



Note: The last bin includes the upper edge (120.00), while the earlier bins are left-inclusive and right-exclusive.

For multiple columns

If you discretized several columns, you can loop through them:

 Python



 Run

```
continuous_cols = ["Price", "Quantity"]
binned = disc.fit_transform(df[continuous_cols])

for idx, col in enumerate(continuous_cols):
    df[f"{col}_bin"] = binned[:, idx].astype(int)
    edges = disc.bin_edges_[idx]
    counts = df[f"{col}_bin"].value_counts().sort_index()

    print(f"\n{col}")
    for i in range(len(edges) - 1):
        print(f"  Bin {i}: [{edges[i]:.2f}, {edges[i+1]:.2f}) -> {counts.get(i,0)} items")
```



Python



Run

Display as a table

For a cleaner output:

Python



Run

```
edges = disc.bin_edges_[0]
counts = df["Price_bin"].value_counts().sort_index()

bin_summary = pd.DataFrame({
    "Bin": range(len(edges)-1),
    "Range": [f"[{edges[i]:.2f}, {edges[i+1]:.2f})"
              for i in range(len(edges)-1)],
    "Count": [counts.get(i,0) for i in range(len(edges)-1)]
})

print(bin_summary)
```

Output:

	Bin	Range	Count
	0	[5.00, 16.50)	2
	1	[16.50, 32.50)	2
	2	[32.50, 67.50)	2
	3	[67.50, 120.00)	2

This table clearly shows each bin's numerical interval and the number of observations assigned to it.



Code Implemented

```
print("Uniform")
print(df_uniform[["UnitPrice"]].value_counts().sort_index())
```

[28] ✓ 0.0s

... Uniform
UnitPrice

0.0	2
2.0	541820
3.0	61
4.0	16
5.0	6
6.0	3
11.0	1

Name: count, dtype: int64

Joseph Fitchet

Joseph's Ai log is in the submission folder as "Jf-Ai_ChatLog.pdf" as it is long and AI generated.